

Go scheduler deep dive

Путь одной goroutine через runtime/proc.go

```
newproc → runqput → schedule → findRunnable → execute  
→ gopark / goready → runqsteal
```

Главный вопрос

Что реально происходит после `go worker()`?

- Кто создаёт `G` и куда она попадает?
- Как `scheduler` выбирает следующую `goroutine`?
- Что происходит при блокировке и пробуждении?
- Как один `P` забирает работу у другого?

Полный pipeline новой goroutine

```
go worker()
```

↓

```
newproc → newproc1 → _Grunnable → runqput
```

↓

```
schedule → findRunnable → execute
```

↓

```
_Grunning → worker()
```

Создание и запуск — разные фазы: новая G сначала становится runnable.

go f() ≠ немедленный запуск

```
func main() {  
  go worker()  
}
```

Компилятор подводит вызов к runtime.newproc.

Runtime создаёт/переиспользует G, переводит её в runnable и кладёт в очередь.

Когда она реально получит CPU — решит scheduler.

Почему runtime использует system stack

user G stack

↓ systemstack

M.g0 system stack

↓

создание G / scheduler metadata / run queue operations

Низкоуровневые операции scheduler выполняются на системном stack M.g0.

newproc1: G можно переиспользовать

```
go worker()
```

```
↓
```

```
gfget(P): есть свободная G?
```

```
├ yes → reuse G
```

```
└ no  → allocate G + stack
```

```
↓
```

```
prepare execution context
```

```
↓
```

```
status = _Grunnable
```

Завершившиеся G runtime держит в пуле и переиспользует.

P: runnext + local run queue

P0

└─ runnext → G42

└─ runq[256] → G10 G11 G12 ...

runnext – кандидат на быстрый следующий запуск.

runq – локальная кольцевая очередь runnable goroutines.

Локальная очередь уменьшает глобальный contention.

Зачем существует `runnext`

Parent создаёт child:

```
go child()
```

Если child отправить в хвост очереди из сотен G – растёт `scheduling latency`.

`runnext` позволяет связанным goroutines быстрее передавать выполнение.

Это `latency optimization`, а не строгий `priority mechanism`.

Переполнение local runq

P0 local runq: ~256 G

+ новая G

↓

runqputslow

↓

часть (~половина) остаётся локально

часть batch-ем уходит в global queue

Так перегруженный P разгружается, а работа становится доступна другим P.

waker: почему нельзя будить thread на каждую G

Каждый park/unpark OS thread имеет цену.

Если будить M на каждую новую G:

work появилась → wake thread → work уже забрал другой M → thread снова sleep.

Runtime будит worker консервативно: нужен idle P и не должно быть уже достаточного spinning M.

Spinning M

SPINNING M

↓

активно ищет работу:

local / global / netpoll / steal

↓

не нашёл

↓

PARK

Сразу park → дорогие wakeups.

Вечно spin → пустая трата CPU.

Spinning — компромисс между latency и CPU.

schedule(): один раунд scheduler

```
schedule()
```

```
↓
```

```
findRunnable()
```

```
↓
```

```
execute(gp, inheritTime)
```

Смысл `schedule` прост: найти runnable `G` и передать ей исполнение.

Большая часть сложности спрятана в поиске работы и безопасных переходах состояний.

findRunnable(): работа не в одной очереди

findRunnable() ищет среди:

1. runtime / GC / timers
2. periodic global fairness check
3. local runq
4. global runq
5. non-blocking netpoll
6. work stealing
7. blocking netpoll / park

Точный порядок — implementation detail runtime.

Locality vs fairness

Local runq:

- + минимум shared coordination
- + cache locality

Но бесконечное предпочтение local work может голодать global queue.

Поэтому runtime периодически проверяет global work раньше local.

Scheduler постоянно балансирует locality и fairness.

Global queue забирается batch-ем

GLOBAL QUEUE

G1 G2 G3 G4 G5 ...

↓ take batch

P2 получает:

- одну G для немедленного execute
- остальные — в local runq

Один доступ к global queue амортизируется: дальше P снова работает локально.

Netpoller перед тяжёлым stealing

Local empty. Global empty.

Kernel мог уже сделать socket readable/writable.

Неблокирующий netpoll возвращает G, которые стали runnable.

Получить готовую I/O goroutine часто дешевле, чем обходить другие P в поиске чужих очередей.

Work stealing

До:

P0: G1 G2 G3 G4 G5 G6 G7 G8

P1: —

P1 идёт steal у P0

После примерно:

P0: G1 G2 G3 G4

P1: G5 G6 G7 + одна G в execute

Пустой P не должен простаивать, пока где-то есть runnable work.

Почему крадём примерно половину

Украсть одну G:

→ быстро снова придётся steal

→ scheduler overhead

Украсть всё:

→ victim сам останется без work

Половина — компромисс: thief получает запас работы, victim сохраняет locality и progress.

execute(): G получает M

```
findRunnable вернул gp
↓
execute(gp)
↓
M.curg = G
G.m = M
_Grunnable → _Grunning
↓
gogo(&G.sched)
↓
восстановить PC/SP/register context
↓
продолжить user code
```

Scheduler не интерпретирует каждую инструкцию

После `gogo` CPU просто исполняет `user code`.

Scheduler возвращается в игру, когда возникает событие:

- `blocking`
- `syscall`
- `preemption`
- `Gosched`
- `stack growth`
- `panic / return`

Scheduler — `event-driven machinery`, а не `interpreter`.

gopark: goroutine больше не может работать

G42 RUNNING

↓

gopark

↓

switch to g0 / park_m

↓

_Grunning → _Gwaiting

↓

detach G from M

↓

schedule another G

Block G ≠ block OS thread для runtime-aware primitives.

goready: событие произошло

event happened

↓

goready(G42)

↓

ready()

↓

_Gwaiting → _Grunnable

↓

runqput

↓

wakeup if needed

Важно: goready не запускает G немедленно — она снова только runnable.

Channel wait: полный lifecycle

```
G1 RUNNING
↓ receive from empty channel
register waiter
↓
gopark → G1 WAITING
↓
M/P executes G2
↓
G2 sends value
↓
goready(G1) → RUNNABLE
↓
scheduler → execute
↓
G1 RUNNING, receive returns
```

Почему park и waiter должны быть согласованы

Опасная гонка:

G1: check → empty

[context switch]

G2: send → no waiter → done

G1: park forever

Это lost wakeup.

Blocking primitive должен согласовать регистрацию waiter и переход в sleep так, чтобы событие невозможно было потерять.

Gosched / preemption $\neq$ park

gopark:

RUNNING $\rightarrow$ WAITING

G сейчас НЕ МОЖЕТ продолжать

Gosched / preemption:

RUNNING $\rightarrow$ RUNNABLE

G МОЖЕТ работать, но уступает execution resource

Это разные причины scheduler switch.

Завершение goroutine

worker() returns

↓

goexit / gdestroy

↓

_Grunning → _Gdead

↓

G может попасть в pool

↓

позже reuse через gfget

Lifecycle объекта G длиннее lifecycle одного вызова goroutine.

Полный lifecycle одной G

```
go f() → newproc → newproc1
↓
RUNNABLE → runqput
↓
schedule → findRunnable → execute
↓
RUNNING
↓ block
gopark → WAITING
↓ event
goready → RUNNABLE
↓
execute → RUNNING
↓ return
goexit → DEAD → pool
```

Что происходит под backend load

100 000 HTTP goroutines $\neq$ 100 000 running tasks.

Например:

20 RUNNING

500 RUNNABLE

70 000 waiting network

20 000 waiting DB pool

9 480 waiting timer/channel/mutex

Диагностика начинается с состояния и wait reason, а не с самого числа goroutines.

Scheduler latency

G стала RUNNABLE в t_0

execute произошёл в $t_0 + 200\text{ ms}$

Это scheduler delay: G готова работать, но execution resource не получила.

Причины:

- CPU saturation
- огромные runnable queues
- GOMAXPROCS / cgroup throttling
- GC/runtime work

Не путать с WAITING на network/DB.

Диагностика: go tool trace

Trace позволяет увидеть:

G created → runnable → running → blocked → runnable → running

Смотрим:

- scheduler latency
- blocking reasons
- network wait
- syscalls
- GC
- P/M activity

Вопрос: handler медленный потому, что работает долго или потому, что долго ждёт CPU?

Что важно запомнить

- go f() создаёт runnable G, а не немедленный thread
- P держит local runq и runnext
- schedule = findRunnable → execute
- local queues дают locality, global queue — coordination/fairness
- work stealing балансирует P
- gopark: RUNNING → WAITING
- goready: WAITING → RUNNABLE
- scheduler управляет execution, но не заменяет backpressure
- под нагрузкой важны RUNNING / RUNNABLE / WAITING и wait reason